

Mini Project: Unity Catalog in Azure Databricks

1. 1. Objective

The objective of this mini project is to explore and implement **Unity Catalog** in Azure Databricks, focusing on its core governance capabilities. By the end of this project, the following goals were achieved:

- **Data Discovery** → making datasets more understandable and searchable.
- **Data Access Control** → applying fine-grained permissions to datasets.
- **Data Audit** → tracking user activities for compliance and monitoring.
- **Data Lineage** → gaining visibility of how data flows and is consumed across Databricks.

Unity Catalog ensures that enterprise data is not only stored and processed but also **secured, governed, and trusted**.

2. Introduction to Unity Catalog

As organizations adopt the **Lakehouse architecture** in Databricks, governance becomes a critical need. Data is often spread across multiple workspaces, teams, and storage accounts, making it hard to manage securely.

- **Unity Catalog** solves this challenge by serving as a **single, centralized governance layer** for all data and AI assets in Azure Databricks.
- Key highlights:
- **Centralized Security** → One place to manage permissions across workspaces.
- **Metadata Management** → Organizes data assets with catalogs, schemas, and tables.
- **Audit & Compliance** → Logs every action for governance.
- **Data Lineage** → Provides visibility from raw data to dashboards.

👉 In short, Unity Catalog is the “**data governance backbone**” of Databricks.

3. Project Setup

3.1 Environment

Platform: Azure Databricks (workspace with Unity Catalog enabled)

Storage: Azure Data Lake Storage Gen2 (ADLS Gen2)

Governance Layer: Unity Catalog

3.2 Unity Catalog Structure

Unity Catalog organizes all assets using a **3-level namespace**:
`catalog.schema.table`

Catalog → The top-level container (e.g., `sales_catalog`).

Schema → Logical grouping of related datasets (e.g., `retail_schema`).

Table → The actual dataset (e.g., `orders`).

This hierarchy ensures clarity and prevents duplication of datasets across teams.

4. Implementation Steps

Step 1 – Enable Unity Catalog

Unity Catalog was enabled in the Databricks workspace through the **Admin Console**.

A **Metastore** (the central metadata store) was created and linked to the workspace.

The Metastore provides governance policies and stores metadata about catalogs, schemas, and tables.

Step 2 – Create Catalogs, Schemas, and Tables

Unity Catalog resources were defined using SQL.

```
-- Create catalog
```

```
CREATE CATALOG sales_catalog;
```

```
-- Create schema inside catalog
```

```
CREATE SCHEMA sales_catalog.retail_schema;
```

```
-- Create a managed table
CREATE TABLE sales_catalog.retail_schema.orders (
  order_id INT,
  customer_id INT,
  product STRING,
  amount DOUBLE,
  order_date DATE
);
```

Here, the **catalog** → **schema** → **table** hierarchy ensures a structured governance model.

Step 3 – Data Access Control

Permissions were assigned to user groups, ensuring secure access.

```
-- Grant table read access to analysts
GRANT SELECT ON TABLE sales_catalog.retail_schema.orders TO
analyst_group;
```

```
-- Revoke access from interns
REVOKE SELECT ON TABLE sales_catalog.retail_schema.orders FROM
interns;
```

Role-based access control was applied, integrating with **Azure Active Directory groups**.

Fine-grained control could also be extended to **column-level** and **row-level**.

Step 4 – Data Discovery

To make datasets more meaningful, **comments and tags** were added to describe tables and columns:

```
COMMENT ON TABLE sales_catalog.retail_schema.orders IS 'Order
details including customers, products, and amounts';
COMMENT ON COLUMN sales_catalog.retail_schema.orders.amount IS
'Transaction value in USD';
```

This enhanced discoverability, allowing users to search for datasets by metadata instead of raw names.

Step 5 – Data Auditing

Unity Catalog automatically **records query history and access logs**.

Logs show **who queried which dataset, what time, and with what action**.

These logs are useful for:

- Compliance** (e.g., GDPR, HIPAA, SOX).
- Security monitoring** to detect unusual data access.

Step 6 – Data Lineage

Unity Catalog automatically captured **end-to-end lineage**:

Where the dataset originated (source system).

How it was transformed in ETL processes.

Which notebooks, dashboards, or models used it.

This helps in:

- Debugging data pipelines.
- Understanding downstream impact before modifying data.
- Ensuring accountability and trust in reports.

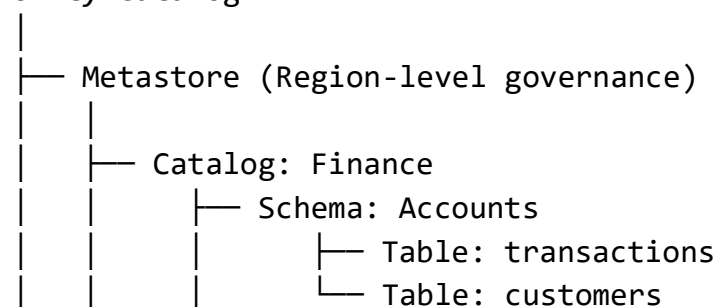
5. Outcomes

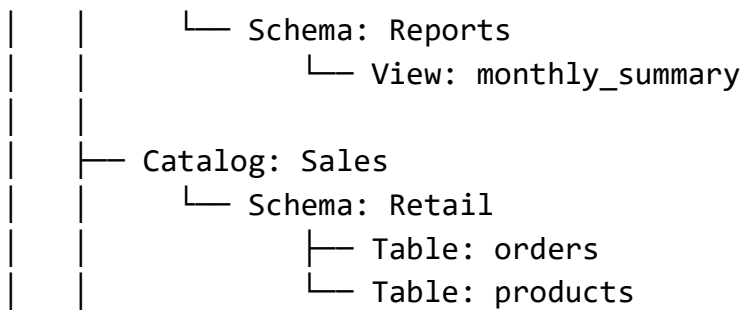
From the implementation, the following outcomes were achieved:

- A **governed structure** (catalog → schema → table) was successfully created.
- **Secure data access** was implemented with role-based policies.
- **Improved discoverability** was achieved with comments and tags.
- **Audit trails** were captured, providing compliance-ready logging.
- **Lineage tracking** gave visibility into the complete lifecycle of data.

9. Architecture Diagram (Simple View)

Unity Catalog





10. Conclusion

Unity Catalog in Azure Databricks is a **powerful governance solution** for the Lakehouse platform. By centralizing permissions, tracking data lineage, and enabling secure sharing, it ensures that:

- Data remains **secure, compliant, and well-managed**.
- Teams can **collaborate efficiently without data duplication**.
- Organizations can **scale governance across multiple clouds and workspaces**.

In short: **Unity Catalog = One place to govern all your data and AI assets in Databricks.**